**Practical 3: Write a program to implement word count program using MapReduce**.

The Word Count program is essentially the "Hello World" of Big Data. It reads text files and counts how often each word occurs. It perfectly demonstrates the two phases of MapReduce:

| Phase | What it does in Word Count |
|---|---|
| **Mapper** | Takes a line of text, splits it into words, and outputs a key-value pair for each word (e.g., <"hello", 1>). |
| **Reducer** | Gathers all the identical keys (words) and adds their values together (e.g., <"hello", [1, 1, 1]> becomes <"hello", 3>). |

Here is the step-by-step manual to write, compile, and execute this on your Hadoop cluster.

## Step 1: Write the Java Program

1. Create a new folder on your C: drive to keep things organized, for example, `C:\HadoopPracticals\WordCount`.
2. Inside that folder, create a new text file and name it `WordCount.java`.
3. Open `WordCount.java` in Notepad or your preferred code editor, and write the following standard MapReduce code in it:

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  public static class TokenizerMapper extends Mapper<Object, Text,
Text, IntWritable>{
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context) throws
IOException, InterruptedException {
      StringTokenizer itr = new StringTokenizer(value.toString());
      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }
```

```
  public static class IntSumReducer extends
Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values, Context
context) throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      context.write(key, result);
    }
  }

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

## Step 2: Compile and Create the JAR File

Hadoop is a Java framework, so we need to compile this code and package it into a `.jar` archive so Hadoop can distribute it across your cluster.

1. Open **Command Prompt as Administrator**.
2. Navigate to your project folder:

   ```
   cd C:\HadoopPracticals\WordCount
   ```

3. Set a temporary variable for the Hadoop classpath (this tells Java where all the Hadoop libraries are):

   ```
   FOR /F "delims=" %I IN ('hadoop classpath') DO set HADOOP_CP=%I
   ```

4. Compile the Java file:

   ```
   javac -cp "%HADOOP_CP%" WordCount.java
   ```

5. Package the compiled `.class` files into a JAR file named `wc.jar`:

   ```
   jar -cvf wc.jar *.class
   ```

## Step 3: Prepare the Input Data in HDFS

MapReduce reads data from the Hadoop Distributed File System (HDFS), not your local Windows C: drive. We need to create a test file and upload it.

1. Create a local text file named `input.txt` in your `C:\HadoopPracticals\WordCount` folder and type some sample text into it (e.g., "hello hadoop hello world big data").
2. Ensure your Hadoop cluster is running (NameNode and DataNode must be active).
3. In your Command Prompt, create an input directory in HDFS:

   ```
   hdfs dfs -mkdir /input
   ```

4. Upload your local text file into the HDFS `/input` directory:

   ```
   hdfs dfs -put input.txt /input/
   ```

## Step 4: Execute the MapReduce Job

Now we tell Hadoop to run our packaged program on the data sitting in HDFS.

1. Run the following command. The syntax is `hadoop jar <jar_name> <main_class> <hdfs_input_path> <hdfs_output_path>`:

   ```
   hadoop jar wc.jar WordCount /input /output
   ```

   *(Note: Do not create the* `/output` *folder beforehand. MapReduce requires that the output directory does not exist before running, or it will throw an error to prevent overwriting old data.)*

   You will see a lot of text scrolling by as the JobTracker assigns the Map and Reduce tasks. Look for the line that says `map 100% reduce 100%` to know it finished successfully.

## Step 5: View the Results

1. Once the job completes, check the contents of the newly created output directory in HDFS:

   ```
   hdfs dfs -ls /output
   ```

2. You will see a file named `part-r-00000`. This contains your final results. Read it using the `-cat` command:

   ```
   hdfs dfs -cat /output/part-r-00000
   ```

   If your output displays a list of words next to the number of times they appeared, you have successfully completed Practical 3.